

改进后的完整软件过程定义文档

项目名称：简单物流信息网站

学生：官若愚 | 学号：2023141461070

日期：2026年6月

版本：V2.0 (AI增强版)

基于：V1.0 (原始软件过程定义) + AI技术应用匹配报告 + 改进方案设计 + 原型验证

第一部分：过程概述与目标

1.1 项目背景

简单物流信息网站是一个小型Web项目，面向普通用户提供订单查询和物流轨迹跟踪服务。项目由3人小团队开发，开发周期约4-6周。项目功能范围明确、技术栈成熟（Java Spring Boot + MySQL + 原生前端），适合作为AI增强软件过程的验证项目。

1.2 过程设计目标

目标维度	V1.0 (原始)	V2.0 (AI增强版)
过程完整性	覆盖6个核心阶段	同V1.0，每个阶段嵌入AI节点
执行效率	手工驱动	AI辅助，整体效率提升40%+
质量保证	人工测试为主	AI+人工双重质量保障
团队赋能	3人分工	3人+AI角色（AI训练师、AI测试分析师）
可度量性	基础度量	增加AI贡献度、效率提升等AI专项指标

1.3 过程设计依据

本过程定义基于以下标准和实践：

来源	应用说明
ISO/IEC 12207	采用其软件生命周期过程框架（规划→开发→验证→交付→维护）
ISO/IEC 15504	参考过程评估思想，确保过程可执行、可检查
ISO/IEC 33001	参考过程改进理念，支持后续迭代优化
智能化软件工程标准	参考中国信通院2024年发布的智能测试能力要求
项目实践	基于3人小团队、短周期的实际约束

1.4 过程剪裁说明

基于V1.0的剪裁原则，V2.0继续保持轻量化：

剪裁项	V1.0处理	V2.0处理
重型管理流程	省略配置管理、风险管理、变更控制	同V1.0，AI辅助简化版风险评估
冗余文档	只保留核心交付物	AI自动生成文档，保持精简
多级评审	合并过渡过程	AI代码审查替代部分人工评审
核心过程	保留6阶段	保留6阶段，各阶段嵌入AI节点

第二部分：角色职责定义

2.1 角色总览

```
graph TD
    PM["项目经理（人）"] --- Dev["开发工程师（人）"]
    PM --- Test["测试工程师（人）"]
    PM --- AI_Trainer["AI训练师（人+AI）"]
    Test --- AI_Analyst["AI测试分析师（人+AI）"]
    Dev --- Claude_Code["Claude Code（AI Agent）"]
```

Dev --- Copilot["GitHub Copilot (AI助手)"]
Test --- Claude["Claude (AI测试生成)"]

2.2 角色职责详述

2.2.1 项目经理（1人）

维度	V1.0职责	V2.0新增职责
核心职责	项目规划、进度管理、需求确认	AI工具选型决策、AI风险管控
交付物	项目计划、进度报告	AI工具使用评估报告
与AI协作	无	审核AI生成的项目评估、利用AI辅助进度预测
技能要求	软件项目管理基础	了解AI工具能力边界

2.2.2 开发工程师（1-2人）

维度	V1.0职责	V2.0新增职责
核心职责	前后端代码开发、数据库搭建	AI生成代码审核与调整、项目Prompt维护
交付物	可运行代码	AI辅助生成的代码+注释
与AI协作	无	使用Claude Code生成代码框架，使用Copilot日常补全
技能要求	Java、Spring Boot、前端	Prompt工程基础、AI代码审查技巧

2.2.3 测试工程师（1人，可兼任）

维度	V1.0职责	V2.0新增职责
核心职责	功能测试、兼容性测试、缺陷记录	AI测试用例审核、AI缺陷预测分析
交付物	测试记录、问题清单	AI辅助测试报告、覆盖率分析
与AI协作	无	使用Claude生成测试用例，分析AI缺陷预测
技能要求	软件测试基础	了解AI测试工具和方法

2.2.4 AI训练师（新增角色，兼任）

维度	说明
角色定位	项目级AI工具管理者和Prompt维护者
核心职责	建立和维护项目专属Prompt库；调优AI输出质量；维护项目上下文文档（CLAUDE.md）；跟踪AI工具版本更新
工作量	每周约2-3小时
关键技能	LLM基础原理、Prompt工程、项目技术栈

2.2.5 AI测试分析师（新增角色，兼任）

维度	说明
角色定位	AI测试质量的管理者
核心职责	审核AI生成测试用例的充分性；补充领域特定的测试场景；分析AI缺陷预测报告；持续优化测试Prompt

维度	说明
工作量	每周约3-4小时
关键技能	测试方法论、业务领域知识、AI测试工具使用

2.3 责任分配矩阵（RACI）

活动	项目经理	开发工程师	测试工程师	AI训练师	AI测试分析师	Claude Code	Copilot
需求获取	R	C	C	S	-	S	-
需求AI分析	A	R	C	S	-	S	-
架构设计	A	R	C	S	-	S	-
AI生成设计图	A	R	C	S	-	S	-
代码开发	-	R	-	S	-	S	S
AI代码审查	-	R	-	-	-	S	-
人工代码审查	-	R	C	-	-	-	-
测试计划	A	C	R	-	S	-	-
AI测试生成	-	-	R	-	S	S	-
测试执行	-	C	R	-	-	-	-
缺陷分析	-	C	R	-	S	S	-
部署上线	R	R	C	-	-	S	-

活动	项目经理	开发工程师	测试工程师	AI训练师	AI测试分析师	Claude Code	Copilot
运维监控	R	R	-	-	-	S	-

R=负责, A=审批, C=咨询, S=AI支持

第三部分：各阶段详细定义

阶段一：需求获取与分析过程（AI增强版）

```
flowchart TD
    Start["项目启动"] --> G1["收集用户原始需求"]
    G1 --> A1["🤖 AI Step 1: 结构化需求提取"]
    A1 --> A1_Review{"人工审核AI提取结果"}
    A1_Review -->|通过| A2["🤖 AI Step 2: 需求一致性验证"]
    A1_Review -->|需修改| A1_Adjust["人工调整后重新提交AI"]
    A1_Adjust --> A1
    A2 --> A2_Review{"人工确认验证结果"}
    A2_Review -->|发现矛盾| A1_Adjust
    A2_Review -->|通过| A3["🤖 AI Step 3: 自动生成验收标准"]
    A3 --> A3_Review{"人工调整验收标准"}
    A3_Review --> A4["输出：需求规格说明书"]
```

过程描述：

要素	说明
输入	用户访谈记录、项目背景文档、竞品参考
AI参与节点	需求结构化、一致性验证、验收标准生成
人工决策节点	需求优先级排序、领域知识补充、最终确认
AI工具	Claude (LLM)

要素	说明
输出	需求规格说明书、用户故事地图、验收标准清单
输出格式	Markdown文档
质量度量	AI提取完整率>90%、误报率<10%、人工调整比例<20%
预估耗时	0.5天 (V1.0: 2天, 效率提升75%)

阶段二：软件设计过程（AI增强版）

```
graph TD
    B0["需求规格说明书"] --> B1["🤖 AI Step 1: 技术方案推荐"]
    B1 --> B1_Review{"人工选择技术方案"}
    B1_Review --> B2_Parallel["并行AI设计生成"]
    B2_Parallel --> B2a["🤖 AI: 数据库ER图生成"]
    B2_Parallel --> B2b["🤖 AI: 页面结构设计"]
    B2_Parallel --> B2c["🤖 AI: API接口规范生成"]
    B2a --> B3["人工设计评审"]
    B2b --> B3
    B2c --> B3
    B3 --> B3_Decision{"评审通过?"}
    B3_Decision -->|通过| B4["输出: AI辅助设计文档包"]
    B3_Decision -->|需修改| B5["人工标注修改意见"]
    B5 --> B2_Parallel
```

过程描述：

要素	说明
输入	需求规格说明书
AI参与节点	技术方案推荐、ER图生成、页面设计、API规范生成

要素	说明
人工决策节点	技术方案选择、设计评审、最终确认
AI工具	Claude + Mermaid (图表生成)
输出	数据库ER图 (Mermaid格式)、页面原型描述、API接口规范 (OpenAPI格式)、设计说明文档
输出格式	Markdown文档 (含Mermaid图表)
质量度量	AI生成设计可用率>80%、人工修改比例<30%
预估耗时	1天 (V1.0: 3天, 效率提升67%)

物流网站设计示例：

数据库ER图 (AI生成Mermaid)：

```
erDiagram
    ORDER ||--o{ TRACKING_NODE : has
    ORDER {
        string order_id PK
        string sender_phone
        string receiver_phone
        string receiver_address
        string status
        datetime created_at
        datetime updated_at
    }
    TRACKING_NODE {
        int node_id PK
        string order_id FK
        string location
        string description
        string status
        datetime timestamp
    }
```


API接口规范示例 (AI生成)： | 方法 | 路径 | 功能 | 请求参数 | 响应 | |-----|-----|-----|
|-----|-----| | GET | /api/orders/{orderId} | 按订单号查询 | orderId: string |
OrderResponse JSON | | GET | /api/orders/track | 按手机号查轨迹 | phone: string |
TrackingNode[] JSON |

阶段三：编码实现过程 (AI增强版)

```
flowchart TD
    C0["设计文档"] --> C1["🤖 AI Step 1: 生成代码框架"]
    C1 --> C2["人工：实现核心业务逻辑"]
    C2 --> C3["🤖 AI Step 2: Copilot辅助补全"]
    C3 --> C4["本地调试运行"]
    C4 --> C4_Check{"功能正常?"}
    C4_Check -->|否| C2
    C4_Check -->|是| C5["🤖 AI Step 3: Claude Code代码审查"]
    C5 --> C5_Review{"审查通过?"}
    C5_Review -->|否| C6["根据AI建议修复"]
    C6 --> C5
    C5_Review -->|是| C7["🤖 AI Step 4: 生成代码注释"]
    C7 --> C8["人工最终审查"]
    C8 --> C9["输出：可运行代码 + 自动文档"]
```

过程描述：

要素	说明
输入	设计文档、API规范、ER图
AI参与节点	代码框架生成、代码补全、代码审查、注释生成
人工决策节点	核心逻辑实现、审查结果判断、最终质量把关
AI工具	Claude Code（主力Agent）、GitHub Copilot（日常补全）
输出	可运行的前后端代码、单元测试代码、代码注释
质量度量	测试通过率>95%、AI代码审查通过率>85%、SonarQube质量门禁通过

要素	说明
预估耗时	5天 (V1.0: 8天, 效率提升38%)

开发规范：

规范项	要求
命名规范	遵循Java/JavaScript标准命名约定
注释要求	AI自动生成基础注释，核心逻辑由人工补充注释
安全要求	禁止SQL拼接，使用参数化查询；前端输出转义防XSS
代码复杂度	单方法不超过50行，圈复杂度不超过10
AI生成代码标识	在AI生成的文件头部添加 <code>@AI-generated</code> 注释标记

阶段四：测试验证过程（AI增强版）

flowchart TD

D0["功能代码"] --> D1["🤖 AI Step 1: 分析代码→生成测试大纲"]
D1 --> D2["人工：确认测试策略"]
D2 --> D3["🤖 AI Step 2: 生成单元测试用例"]
D3 --> D4["🤖 AI Step 3: 生成集成测试用例"]
D4 --> D5["🤖 AI Step 4: 生成前端自动化测试"]
D5 --> D6["执行测试套件"]
D6 --> D6_Check{"全部通过?"}
D6_Check -->|否| D7["🤖 AI Step 5: 分析失败原因"]
D7 --> D8["人工修复 / AI建议修复"]
D8 --> D6
D6_Check -->|是| D9["🤖 AI Step 6: 预测未覆盖缺陷"]
D9 --> D10["人工：补充探索性测试"]
D10 --> D11["🤖 AI Step 7: 生成测试报告"]
D11 --> D12["输出：完整测试报告"]

过程描述：

要素	说明
输入	功能代码、需求规格说明书
AI参与节点	测试大纲生成、测试用例生成、失败分析、缺陷预测、报告生成
人工决策节点	测试策略确认、补充领域测试、缺陷修复决策
AI工具	Claude（测试生成）、JUnit 5 + Mockito（单元测试框架）、Selenium（前端测试）、JaCoCo（覆盖率）
输出	测试用例代码、测试执行报告、覆盖率报告、缺陷清单
质量度量	行覆盖率>90%、分支覆盖率>85%、缺陷检出率>80%
预估耗时	1.5天（V1.0: 3天，效率提升50%）

测试类型与策略：

测试类型	覆盖目标	AI生成比例	人工补充重点
单元测试	Service层业务逻辑	80% AI生成	复杂业务规则验证
集成测试	Controller + Repository	70% AI生成	数据库事务边界测试
前端测试	页面交互流程	60% AI生成	跨浏览器兼容性
探索性测试	非预期使用场景	AI建议测试方向	人工自由探索

阶段五：部署与交付过程（AI辅助版）

```
graph LR
    E0["测试通过版本"] --> E1["🤖 AI：生成部署脚本"]
    E1 --> E2["🤖 AI：生成使用说明"]
    E2 --> E3["本地部署验证"]
    E3 --> E4["🤖 AI：辅助部署验证"]
    E4 --> E4_Check{"验证通过?"}
    E4_Check -- 否 --> E5["修复部署问题"]
```

E5 --> E3

E4_Check -->|是| E6["服务器部署"]

E6 --> E7["交付可用版本"]

过程描述：

要素	说明
输入	测试通过的代码版本
AI参与节点	部署脚本生成、使用说明编写、部署验证辅助
人工决策节点	部署执行、功能验证、最终交付确认
AI工具	Claude（脚本和文档生成）
输出	部署包（WAR/JAR）、部署说明文档、使用说明文档
质量度量	部署成功率>95%、文档完整性>90%
预估耗时	0.5天（V1.0: 1天，效率提升50%）

阶段六：维护与优化过程（AI辅助版）

flowchart TD

F0["线上版本运行"] --> F1["监控运行状态"]

F1 --> F1_Check{"发现异常?"}

F1_Check -->|否| F1

F1_Check -->|是| F2["🤖 AI Step 1: 日志异常模式识别"]

F2 --> F3["🤖 AI Step 2: 根因分析"]

F3 --> F4["🤖 AI Step 3: 建议修复方案"]

F4 --> F5["人工评估修复方案"]

F5 --> F6["实施修复"]

F6 --> F7["🤖 AI: 辅助回归验证"]

F7 --> F8["发布优化版本"]

过程描述：

要素	说明
输入	线上运行日志、用户反馈、监控数据
AI参与节点	异常检测、根因分析、修复方案建议、回归验证
人工决策节点	修复方案评估、优先级排序、最终发布决策
AI工具	Claude（日志分析、方案建议）
输出	Bug修复版本、优化版本
质量度量	故障响应时间<2小时、修复方案可用率>80%
预估耗时	按需投入，AI减少分析诊断时间50%

第四部分：交付物清单

4.1 各阶段交付物

阶段	交付物	格式	AI参与度	负责人
需求分析	需求规格说明书	Markdown	80%（AI生成初稿+人工审核）	项目经理
需求分析	用户故事地图	Markdown表格	90%（AI生成+人工调整）	项目经理
需求分析	验收标准清单	Markdown	85%（AI生成+人工审核）	项目经理
软件设计	数据库ER图	Mermaid代码	90%（AI生成+人工确认）	开发工程师
软件设计	API接口规范	OpenAPI/Markdown	85%（AI生成+人工调整）	开发工程师

阶段	交付物	格式	AI参与度	负责人
软件设计	设计说明文档	Markdown	70% (AI生成框架+人工补充)	开发工程师
编码实现	后端Java源代码	.java文件	60% (AI生成+人工实现)	开发工程师
编码实现	前端页面源代码	.html/.js/.css	55% (AI生成+人工实现)	开发工程师
编码实现	单元测试代码	.java文件	80% (AI生成+人工审核)	开发工程师
测试验证	测试执行报告	Markdown	70% (AI生成+人工分析)	测试工程师
测试验证	覆盖率报告	HTML (JaCoCo生成)	100% (工具自动生成)	测试工程师
测试验证	缺陷清单	Markdown表格	60% (AI整理+人工确认)	测试工程师
部署交付	部署包	WAR/JAR	100% (构建工具+AI脚本)	开发工程师
部署交付	部署说明	Markdown	90% (AI生成+人工审核)	开发工程师
部署交付	使用说明	Markdown	90% (AI生成+人工审核)	项目经理
维护优化	问题修复记录	Markdown	50% (AI辅助+人工主导)	开发工程师

4.2 文档模板说明

所有交付文档均采用Markdown格式，遵循以下模板结构：

```
# 文档标题
> 元信息：项目名称、版本、日期、作者

## 1. 概述
## 2. 详细内容
## 3. 结论
```

第五部分：过程度量指标

5.1 核心度量指标

```
flowchart LR
    subgraph Efficiency["效率指标"]
        E1["需求周期时间"]
        E2["编码周期时间"]
        E3["测试周期时间"]
        E4["AI辅助效率提升率"]
    end

    subgraph Quality["质量指标"]
        Q1["测试覆盖率"]
        Q2["缺陷密度"]
        Q3["代码复杂度"]
        Q4["AI生成代码可用率"]
    end

    subgraph AI_Specific["AI专项指标"]
        AI1["AI输出接受率"]
        AI2["AI辅助节省时间"]
        AI3["AI输出质量评分"]
        AI4["AI工具ROI"]
    end
```

5.2 指标定义与目标值

类别	指标	定义	计算方式	目标值
效率	需求周期时间	从需求获取到需求确认的总耗时	需求阶段总工时	≤0.5天
效率	编码周期时间	从设计完成到代码可测试的总耗时	编码阶段总工时	≤5天
效率	AI辅助效率提升率	AI辅助相对传统方式的效率提升比例	$(\text{传统耗时} - \text{AI耗时}) / \text{传统耗时} \times 100\%$	≥40%
质量	测试覆盖率	代码被测试覆盖的比例	JaCoCo行覆盖率	≥90%
质量	分支覆盖率	代码分支被测试覆盖的比例	JaCoCo分支覆盖率	≥85%
质量	缺陷密度	每千行代码中的缺陷数	缺陷数/KLOC	≤2
质量	AI生成代码可用率	AI生成代码无需修改可直接使用的比例	$\text{可用的AI代码行} / \text{总AI代码行} \times 100\%$	≥70%
AI专项	AI输出接受率	人工审核后接受的AI输出比例	$\text{接受的AI输出} / \text{总AI输出} \times 100\%$	≥80%
AI专项	AI辅助节省时间	AI工具帮助节省的总工时	传统方式估算-实际AI辅助耗时	≥20小时
AI专项	AI工具ROI	AI工具投入产出比	$\text{节省工时} \times \text{人时成本} / \text{AI工具费用}$	≥3:1

5.3 度量数据收集

指标	收集方式	收集频率	分析工具
阶段耗时	开发者工时记录	每日	Toggl/手动记录
测试覆盖率	JaCoCo自动生成	每次构建	JaCoCo + SonarQube
代码质量	SonarQube静态分析	每次代码提交	SonarQube

指标	收集方式	收集频率	分析工具
AI接受率	人工标注（接受/修改/拒绝）	每次AI交互	手动记录+统计
AI节省时间	对比估算	每个阶段完成时	手动统计

第六部分：AI工具使用指南

6.1 项目AI工具栈

工具	版本/型号	用途	使用场景	费用
Claude Code	latest	代码生成、审查、测试生成	全阶段	API调用计费
GitHub Copilot	latest	实时代码补全	日常编码	\$10/月/人
Claude (Web/API)	deepseek-v4-pro	需求分析、文档生成、设计	需求/设计/文档	API调用计费
SonarQube	社区版	静态代码分析	编码阶段	免费
JaCoCo	latest	测试覆盖率统计	测试阶段	免费
JUnit 5	5.x	单元测试框架	测试阶段	免费

6.2 Prompt管理

项目需维护一个Prompt库，存放各阶段经过验证的Prompt模板：

```
项目根目录/
├── .claude/
│   ├── CLAUDE.md           # 项目级上下文文档
│   └── prompts/
│       ├── requirements.md  # 需求分析Prompt模板
│       └── design.md        # 设计Prompt模板
```

	└─── coding.md	# 编码Prompt模板
	└─── testing.md	# 测试Prompt模板

6.3 AI使用规范

规范	说明
输出标注	AI生成的代码文件头部使用 <code>@AI-generated</code> 注释标记
人工审核	AI生成的所有输出必须经过人工审核才能进入下一环节
敏感信息	不在Prompt中包含密码、密钥、用户隐私数据
版本记录	重要的AI交互记录保存为项目文档的一部分
持续优化	每周回顾AI输出质量，更新Prompt模板

第七部分：过程改进记录

版本	日期	改进内容	改进依据	影响
V1.0	2026年4月	原始软件过程定义（6阶段剪裁版）	ISO/IEC 12207/15504/33001	建立基础过程框架
V2.0	2026年6月	AI增强版：需求AI分析、AI辅助设计、AI编码审查、AI测试生成	AI技术调研+原型验证	整体效率提升40%+，质量全面提升

第八部分：总结与展望

本软件过程定义文档V2.0在V1.0的基础上，系统地融入了AI技术元素。通过对6个核心阶段的AI增强改造，新增2个AI相关角色（AI训练师、AI测试分析师），建立了AI输出物质量标准 and 过程度量体系。原型验证结果表明，AI辅助可使编码效率提升54.4%，测试效率提升71.6%。

下一步演进方向： 1. **AI自主Agent：**探索Claude Code Agent模式承担端到端简单功能开发 2. **智能质量门禁：**AI+静态分析+自动化测试形成自动化质量保障体系 3. **预测性过程管理：**利用AI预测项目风险、进度偏差 4. **知识沉淀：**将项目AI协作经验固化为可复用的组织级AI过程资产

参考文献

1. ISO/IEC 12207:2017. *Systems and software engineering — Software life cycle processes*
2. ISO/IEC 15504. *Information technology — Process assessment*
3. ISO/IEC 33001:2015. *Information technology — Process assessment — Concepts and terminology*
4. 中国信通院. 《智能化软件工程技术和应用要求 第3部分:智能测试能力》 (2024)
5. AI4SE行业调查组. 《AI4SE行业现状调查报告》 (2024)
6. GitHub. *GitHub Copilot Documentation* (2024-2026)
7. Anthropic. *Claude Code Documentation* (2025-2026)
8. Gtest全球软件测试技术峰会. 2025会议资料
9. 李华等. 《AI原生自动化测试范式研究》, 软件学报 (2025)
10. 张明等. 《生成式AI在需求工程中的应用》, 计算机科学 (2025)